



SAS[®] Retrieval Agent Manager: Developer's Guide

2026.06

This document might apply to additional versions of the software. Open this document in [SAS Help Center](#) and click on the version in the banner to see all available versions.

Custom Source API Documentation	1
sasram.source	2
Agents API Documentation	3
sasram.agent	3
Code MCP Tool API Documentation	16
sasram.mcp.tool	16
Integrating the Retrieval Agent Manager API with Applications	18
Authentication Flows	18
Preconfigured Python Packages for Agents and Custom Sources	21

Custom Source API Documentation

The SAS Retrieval Agent Manager source client provides access that enables developers to use Python code to create, save, and delete content in a SAS Retrieval Agent Manager source. Agent developers are required to implement the `exec` function in the `run.py` file at the top level of their agent package. This is the entry-point for the custom source.

sasram.source

SourceClient Objects

The helper class that stores query response information.

```
class SourceClient
```

save_file

Saves the file at the given file path to the source.

filepath

The path of the file that you want to save to the source.

```
def save_file(filepath: str) -> None
```

Example:

```
python3
# Create and add content to the file.
temp_file_path = "/tmp/my_first_custom_source.txt"
with open(temp_file_path, "w") as f:
    f.write("Hello world!")

# Save the file to the source.
client.save_file(temp_file_path)
```

delete

Deletes a particular file by name from the source.

filename

The name of the file that you want to delete from the source.

```
def delete(filename: str) -> None
```

Example:

```
python3
client.delete("my_first_custom_source.txt")
```

clear

Removes all files from the source.

```
def clear() -> None
```

Example:

```
python3
client.clear()
```

invoke_llm

Invokes a configured large language model (LLM) using an LLM alias and returns an object that contains the LLM response.

prompt

The query to send to the LLM.

alias

The name of the LLM that you want to use. The name must match an LLM alias defined in SAS Retrieval Agent Manager.

```
def invoke_llm(prompt, alias: str)
```

Returns an object that contains the LLM response.

Example:

```
python3
# Retrieve a document and define the prompt
document = open("/tmp/input.txt").read()
prompt = "Please summarize this document: " + document

#Retrieve the LLM response
invoke_response = client.invoke_llm(prompt, alias="default")
response = invoke_response.response

#Write the response to a file
filename = '/tmp/summary.txt'
with open(filename, 'w') as f:
    f.write(response)

#Save the file
client.clear()
client.save_file(filename)
```

Agents API Documentation

The SAS Retrieval Agent Manager agents client provides developers with easy access to application functions, such as executing queries and retrieving documents. Agent developers are required to implement the `exec` function in the `run.py` file at the top level of their agent package. This is the entry-point for the agent.

Note: For more information about the agent package, see [“About Agents” in SAS Retrieval Agent Manager: User’s Guide](#).

sasram.agent

Agent developers are required to implement the `exec` function in `run.py` at the top level of their agent package.

Client Objects

The helper class that enables an agent to interact with the SAS Retrieval Agent Manager API.

```
class Query(abc.ABC)
```

post_query_async

An asynchronous version of `post_query`. To get the response, call the blocking `get_result` method on the returned `QueryRequest` object.

prompt

The query content either as a string or as a list of LangChain messages

llm_name

The name of LLM to use for query. If provided, the name must match an LLM defined in the plug-in configuration. Otherwise, the 'default' LLM configured for the plug-in is used.

collection_name

The name of the collection to query against. If provided, the name must match a collection defined in the plug-in configuration. Otherwise, the 'default' collection configured for the plug-in is used.

direct_llm_query

The decision of whether to directly query the LLM or augment with RAG.

search_kwargs

Any additional arguments to use for this query. You can specify the top-k or any metadata filters to use for this query. If provided, these arguments override the settings in your prompt template.

persistent

The decision of whether to save the query in the Retrieval Agent Manager database.

tools

A list of tools in LangChain tool format to use for this request.

```
def post_query_async(
    self,
    prompt: Union[str, list[langchain_core.messages.base.BaseMessage]],
    llm_name: Optional[str] = 'default',
    collection_name: Optional[str] = 'default',
    direct_llm_query: bool = False,
    search_kwargs: Optional[dict] = None,
    persistent: bool = True,
    tools: list[dict] = []
) -> QueryRequest:
```

Returns a `QueryRequest` object that can be used to get the result of the query.

Example:

```
query_request = client.post_query_async(
    "What is the capital of France?"
)#
do other things
query_response = query_request.get_result()
if query_response.response is not None:
    print(query_response.response['answer'])
```

post_query

Sends a query request to the SAS Retrieval Agent Manager application and waits for the query to complete.

prompt

The query content either as a string or as a list of LangChain messages

llm_name

The name of LLM to use for query. If provided, the name must match an LLM defined in the plug-in configuration. Otherwise, the 'default' LLM configured for the plug-in is used.

collection_name

The name of the collection to query against. If provided, the name must match a collection defined in the plug-in configuration. Otherwise, the 'default' collection configured for the plug-in is used.

direct_llm_query

The decision of whether to directly query the LLM or augment with RAG.

search_kwargs

Any additional arguments to use for this query. You can specify the top-k or any metadata filters to use for this query. If provided, these arguments override the settings in your prompt template.

persistent

The decision of whether to save the query in the Retrieval Agent Manager database.

tools

A list of tools in LangChain tool format to use for this request.

```
def post_query(
    self,
    prompt: Union[str, list[langchain_core.messages.base.BaseMessage]],
    llm_name: Optional[str] = 'default',
    collection_name: Optional[str] = 'default',
    direct_llm_query: bool = False,
    search_kwargs: Optional[dict] = None,
    persistent: bool = True,
    tools: list[dict] = []
) -> Query:
```

Returns a QueryResponse object with the results of the query.

Example:

```
query_response = client.post_query(
    "What is the capital of France?"
) if query_response.response is not None:
    print(query_response.response['answer'])
```

retrieve_docs

Retrieves documents from a set of collections based on the content provided.

content

The content to use for document retrieval.

collection_name

The name of the collection to fetch documents from. If provided, the name must match a collection defined in the plug-in configuration. Otherwise, the 'default' collection configured for the plug-in is used.

search_kwargs

Any keyword arguments that you want to use for the retriever. At a minimum, you must supply a "k" value, which specifies the number of documents to retrieve. Refer to your vector store's documentation for more information about available arguments.

```
def retrieve_docs(
    self,
    content: str,
    collection_name: Optional[str] = 'default',
    search_kwargs: dict = {'k': 5}
) -> list[langchain_core.documents.base.Document]:
```

Returns a typing.List of langchain_core.documents.base.Document objects.

Example:

```
docs = client.retrieve_docs("What is the capital of France?")
for doc in docs:
    print(doc.page_content)
```

get_session_queries

Gets the queries for the current query session.

```
def get_session_queries(self) -> list[Query]:
```

Returns a list of query objects containing information for each query, such as content, response, and metadata.

Example:

```
queries = client.get_session_queries()
print(queries[0].content)
```

invoke_tool

Invokes the specified tool with the given parameters using the Retrieval Agent Manager API.

```
def invoke_tool(
    self,
    tool_server_id: str,
    tool_name: str,
    args: dict,
    timeout: float
) -> dict:
```

get_langchain_llm

Returns a RAMLLM model for use with LangChain libraries. If the collection_alias passed is None, RAG is not enabled as there is no collection to query against.

llm_alias

The LLM to use for requests. The value must be one of the aliases defined in the agent settings.

collection_alias

The name of the collection to query against. If provided, the name must match a collection defined in the plug-in configuration. Otherwise, the 'default' collection configured for the plug-in is used. It is also possible to pass None if you do not want to query a specific collection.

```
def get_langchain_llm(
    self,
    llm_alias: str = 'default',
```

```
collection_alias: Optional[str] = 'default'
) -> 'RAMLLM':
```

Returns a RAMLLM, which is an instance of `langchain_core.BaseChatModel` for use in your agent.

Example:

```
llm = client.get_ram_llm(llm_alias="default", collection_alias="default")
response = llm.invoke("Hello world")
```

get_langchain_tools

Returns the RAMTools configured for this plug-in for use with LangChain libraries.

tool_alias

The tools that you want to fetch.

```
def get_langchain_tools(self, tool_alias: str = 'default') ->
List[ForwardRef('RAMTool')]:
```

Returns a list of RAMTool, which are instances of `langchain_core.BaseTools` for use in your agent.

Example:

```
tools = client.get_langchain_tools(tool_alias="default")
weather_tool = tools[0]
tool_result = weather_tool.invoke({"search": "weather in Chicago"})
```

get_langgraph_agent

Uses the configured LLM and tools for this agent to create a LangGraph agent.

llm_alias

The LLM to use for requests. The value must be one of the aliases defined in the agent settings.

collection_alias

The name of the collection to query against. If provided, the name must match a collection defined in the plug-in configuration. Otherwise, the 'default' collection configured for the plug-in is used. It is also possible to pass `None` if you do not want to query a specific collection.

tool_alias

The tools that you want to fetch.

```
def get_langgraph_agent(
self,
llm_alias: str = 'default',
collection_alias: Optional[str] = 'default',
tool_alias: str = 'default'
) -> langgraph.graph.state.CompiledStateGraph:
```

Returns a LangGraph `CompiledStateGraph` using the specified tools, collections, and LLMs.

Example:

```
weather_agent = client.get_langgraph_agent(
llm_alias="default",
collection_alias=None,
tool_alias="weather",
)
response = asyncio.run(
weather_agent.ainvoke(
{"messages": "what is the weather in Chicago currently?"})
```

```
))
```

exec

Runs the agent logic. This method is implemented by the agent developer. The string returned is saved as the response to the query.

text

The query content originating from the user.

client

The client object to interact with the Retrieval Agent Manager API.

options

The options passed to the agent.

```
def exec(
    text: str,
    client: Client,
    options: dict[str, typing.Any]
) -> str:
```

Returns output text from the agent. The output text is saved as the `QueryResponse.response['answer']` field in the database.

Example:

```
# run.py
def exec(text, client, options):
    if text == "What is the capital of France?":
        # This one's too easy, don't need an LLM for this
        return "Paris"
    # Use the Client to send a RAG query through our "default"
    # collections and "default" LLM
    query_response = client.post_query(text)
    if query_response.response is not None:
        return query_response.response['answer']
    else:
        return "I'm sorry, I don't know the answer to that."
```

Note: Responses from an agent are rendered as Markdown in the SAS Retrieval Agent Manager chat interface. This means that you can use Markdown syntax to format your response to include tables, Base64 images, lists, and other elements.

For example, consider the following `exec` method.

```
def exec(text, client, options):
    return """
bold italic underline `code`
| Test | Column 1 | Column 2 |
| ---- | - - - - - | - - - - - |
| Row 1 | Lore | Ipsum |
| Row 2 | dolor | si |
| Row 2 | amet | consectetur |
! [Graph] (data:image/png;base64,iVBORw0KGgoAAAANSUheUgAAAGAAAABgCAMAAADVRock
AAAAAXNSR0IArs4c6QAAAAARnQU1BAACxjwv8YQUAAAZUExURQAAAAAAAAAAAAAAAAAAAA
AAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAKMFRskAAAAARGFJOuWd/EIC/36
9g759wQDagUI/PPiQpdwAAAA1wSFlzAAAAOwwAADsMBx2+oZAAAAftJREFUaEPt19tyhCAMQ
A2K4G31/7+2QbMtIEqQ7XSmk/PSuNPhaBKCNoIgcMLnAaDgtxBBFhFkEUGWvxCottO6N3RV
y1llgLOwMdf3JSTACdN08uD/0Sx2xYAQ77sGsod+DSiLBYuFYH0OAF4U1RILeu+0XWEVhBaF
```

```
gBrQtQiHSwU1RBKBiCJTfJnu4hgUBZmCjc6T/Qq4HAWebRQZixZwSCHlqKiC3+oRxfgBmKbn
gCS9FjfMhRvHk1XM0ktfKy5wvac0LMxcBQq2VudF+gwx5yqHSnTtqNw5mubvEESyrhQ6LMU
wewoUDT9S2ewKQeGjuVojcu03acYLuuj48nODXpTrSManH5VuHUHXBW0Y93eIIhme6ozDOe
Ey73A07djrPR/SIn+y4uc3vMcNcQrFn1C9K4ez2h91E+ME69vOBcZuTYk7j1s2dSXpDczdQ
QJl8FhiDRvt9TK3+oMgSJ3fyK5voNDAHWMk5Eessk4QiwZcJHUIJRnwrHcBpIBRniCXAMBN
fc8TPPE0RVwH00/8LEE2AjecO/5AGYgmb1hr8petdgCnDUvscObrLEcLqEK8DJeZRh0Zwh/
QNX4A6CzSglatiKXonZAvfBsNOXvXLzBfhx0oEt/nYrETxCBf1EkEUEWUSQRQRZ/oOgHloq
Df1PFbSUIAiC8DGa5gt4sgjSErUKjwAAAABJRUSErkJggg==)
"""
```

The response is rendered in the chat interface as follows.

bold *italic* underline `code`

Test	Column 1	Column 2
Row 1	Lore	Ipsum
Row 2	dolor	si
Row 2	amet	consectetur



init

This method is called only through the automation pipeline when the agent automation is triggered after the associated collection finishes vectorizing. This is an optional method that is implemented by the agent developer.

client

The client object that interacts with the Retrieval Agent Manager API.

```
def init(client: Client) -> None:
```

Query Objects

The helper class that stores query response information.

```
class Query(pydantic.main.BaseModel)
```

id

The ID of the query in the application database

content

The content of the query.

response

The response of the query. None is returned if there was an error. Check `errors_code` and `errors_text` for more information.

10

feedback

The ID feedback for the query (thumbs up or thumbs down).

duration

The total duration of the query.

configuration_ids

The IDs of the configurations used for the query.

llm_id

The ID of the LLM used for the query.

user_email

The email address of the user who made the query.

metadata

The metadata of the query.

errors_code

The code of an error. 0 is returned if there are no errors.

errors_text

The error message, if there was an error.

prompt_tokens

The number of tokens that are used for the prompt.

prompt_cost

The cost (in USD) of the prompt tokens that are used.

completion_tokens

The number of tokens used for completion.

completion_cost

The cost (in USD) of the completion tokens that are used.

insert_timestamp

The timestamp of the query in ISO-8601 format.

query_session_id

The ID of the query session.

Response Objects

The helper class that holds the response information.

```
class Query.QueryResponse(pydantic.main.BaseModel)
```

input

The input for the response.

answer

The information in the response.

context

The contextual information retrieved during query execution.

Metadata Objects

The helper class that holds the metadata information.

```
class Query.QueryMetadata(pydantic.main.BaseModel)
```

preproc_duration

The time spent by the server in preprocessing.

chat_duration

The time spent by the LLM call.

retrieval_duration

The time spent by the retriever.

model_name

The name of LLM used for the query.

default_prompt_price_for_1k

The prompt price per 1k tokens.

default_completion_price_for_1k

The completion price per 1k tokens.

parent_query_id

The parent query ID for this query.

plugin_id

The plug-in ID for this query.

plugin_name

The plug-in name for this query.

plugin_version

The plug-in version for this query.

chat_init_command

The chat initialization command.

tool_calls

The tool calls selected for this query.

QueryRequest Objects

The helper class that fetches asynchronous query results.

```
class QueryRequest(abc.ABC)
```

get_result

Gets the result of an asynchronous query from the client. Code execution stops until the query finishes or a time-out occurs before proceeding.

timeout

The time-out period in seconds.

```
def get_result(self, timeout: int = 120) -> Query:
```

Returns a QueryResponse object.

AgentStore Objects

The helper class that provides a standard way to create an ABC using inheritance.

```
class AgentStore(abc.ABC)
```

set

Sets a value in the store.

```
def set(self, key: str, value: Any) -> None:
```

get

Gets a value from the store.

```
def get(self, key: str) -> Any:
```

get_all

Gets all values from the store.

```
def get_all(self) -> dict[str, typing.Any]:
```

OrchestratorClient Objects

Enables an orchestrator agent to interact with the SAS Retrieval Agent Manager API and coordinate sub-agents via the A2A protocol.

```
class OrchestratorClient()
```

post_query_async

An asynchronous version of post_query. To get the response, call the blocking get_result method on the returned QueryRequest object. Orchestrator queries are always sent as direct LLM queries. Collection-related parameters and the tools parameter are not available.

prompt

The query content either as a string or as a list of LangChain messages.

llm_name

The name of LLM to use for the query. If provided, the name must match an LLM that is defined in the agent configuration. Otherwise, the default LLM that is configured for the agent is used.

persistent

Indicates whether to save the query in the Retrieval Agent Manager database.

a2a_tools

A list of A2A tool definitions in dictionary format to include with the request.

```
def post_query_async(prompt: Union[str, list[BaseMessage]],
                    llm_name: str = "default",
                    persistent: Optional[bool] = None,
                    a2a_tools: Optional[List[Dict]] = None) -> QueryRequest
```

Returns:

A QueryRequest object that can be used to get the result of the query is returned.

Example:

```

query_request = client.post_query_async(
    "What is the capital of France?"
)
# Do other things.
query_response = query_request.get_result()
if query_response.response is not None:
    print(query_response.response.answer)

```

get_session_queries

Fetches the queries for the current query session.

```
def get_session_queries() -> list[Query]
```

Returns:

The list of Query objects that contain information for each query, such as content, response, and metadata, is returned.

Example:

```

queries = client.get_session_queries()
print(queries[0].content)

```

get_langchain_llm

Returns a RAMLLM model for use with LangChain libraries. RAG is always disabled in the returned RAMLLM since orchestrators do not use collections.

llm_alias

The LLM to use for requests. The value must be one of the aliases defined in the agent settings.

timeout

The time-out period in seconds.

```

def get_langchain_llm(llm_alias: str = "default",
    timeout: int = 120) -> RAMLLM

```

Note: Unlike the standard client's `get_langchain_llm`, the orchestrator version does not accept a `collection_alias` parameter.

get_session_history_as_langchain

Fetches the previous session queries and formats them as LangChain messages for use with LangChain models.

```
def get_session_history_as_langchain() -> list[BaseMessage]
```

Returns:

A list of LangChain BaseMessage objects representing the conversation history is returned.

get_accessible_agent_cards

Returns agent cards for all agents that are accessible to this orchestrator.

```
async def get_accessible_agent_cards() -> list[RAMAgentCard]
```

Returns:

A list of RAMAgentCard instances is returned.

Note: This method is asynchronous and must be awaited.

query_other_agent

Sends a query to another agent via the A2A protocol and returns the response.

agent_card

The target agent's metadata card, obtained from `get_accessible_agent_cards`.

query

The text prompt to send to the agent.

```
async def query_other_agent(agent_card: RAMAgentCard,
                           query: str) -> Union[str, Dict[str, Any]]
```

Returns:

The response from the sub-agent is returned. This is typically a string. If the response cannot be parsed as text, a dictionary with the full payload is returned. If an error occurs, a dictionary with an error key is returned.

Note: This method is asynchronous and must be awaited.

get_a2a_tools

Returns a list of A2ATool objects for querying sub-agents. Each A2ATool is a LangChain tool that wraps an A2A call to a sub-agent.

```
async def get_a2a_tools() -> List[A2ATool]
```

Returns:

A list of A2ATool instances is returned.

Note: This method is asynchronous and must be awaited.

get_langgraph_agent

Uses the configured LLM and the sub-agents accessible to this orchestrator to create a LangGraph agent. The agent is pre-configured with A2ATool instances.

llm_alias

The LLM to use for requests. The value must be one of the aliases defined in the agent settings.

query_timeout

The time-out period in seconds for LLM requests.

```
async def get_langgraph_agent(llm_alias: str = "default",
                              query_timeout: int = 120) -> CompiledStateGraph
```

Returns:

A LangGraph CompiledStateGraph is returned. The agent uses the specified LLM and has access to all sub-agents as A2A tools.

Note: Unlike the standard client's `get_langgraph_agent`, the orchestrator version does not accept `collection_alias` or `tool_alias` parameters. This method is asynchronous and must be awaited.

exec

Implemented by the orchestrator agent developer to run the orchestrator logic. The returned string is saved as the response to the query.

```
async def exec(text: str, ram_client: OrchestratorClient) -> str
```

Note: The orchestrator exec is an asynchronous function. The options parameter is not available for orchestrator agents.

Returns:

Output text from the orchestrator agent is returned. This text is saved as the `QueryResponse.response.answer` field in the database.

Example:

```
# run.py
from langchain_core.messages import HumanMessage, SystemMessage
from sasram.agent import OrchestratorClient

async def exec(text: str, ram_client: OrchestratorClient) -> str:
    agent = await ram_client.get_langgraph_agent()

    session_history = ram_client.get_session_history_as_langchain()
    messages = (
        [SystemMessage("Use your sub-agents to inform your responses.")]
        + session_history
        + [HumanMessage(content=text)]
    )

    response = await agent.ainvoke({"messages": messages})
    return response["messages"][-1].content
```

init

This method is called only through the automation pipeline when the agent automation is triggered after the associated collection finishes vectorizing. This is an optional method that is implemented by the agent developer.

ram_client

The `OrchestratorClient` object to interact with the SAS Retrieval Agent Manager API.

```
async def init(ram_client: OrchestratorClient) -> None
```

RAMAgentCard Objects

The metadata card for a sub-agent accessible to the orchestrator. Extends the A2A protocol's `AgentCard` with an additional `agent_id` field.

```
class RAMAgentCard(AgentCard)
```

agent_id

The unique identifier of the agent in the SAS Retrieval Agent Manager database.

name

The display name of the agent.

description

A description of the agent's capabilities.

skills

A list of skills that the agent supports. Each skill has a name, description, and examples (example queries).

url

The URL of the agent's A2A server endpoint.

A2ATool Objects

A LangChain tool that wraps an A2A call to a sub-agent. Extends RAMTool.

```
class A2ATool(RAMTool)
```

A2ATool is used internally by `get_langgraph_agent` and `get_a2a_tools`. Each A2ATool takes a query string as input and sends it to the corresponding sub-agent via the A2A protocol.

name

The name of the tool, derived from the sub-agent's name.

description

A description of the tool, including the sub-agent's name, description, and skills.

query

The query string to send to the sub-agent.

Example:

```
# Using A2ATools directly (rather than through get_langgraph_agent)
a2a_tools = await client.get_a2a_tools()
weather_tool = next(t for t in a2a_tools if "weather" in t.name.lower())
result = weather_tool.invoke({"query": "What is the weather in Chicago?"})
print(result["response"])
```

Code MCP Tool API Documentation

The SAS Retrieval Agent Manager MCP tool decorator enables developers to expose Python functions as MCP tools. To create MCP tools, developers decorate functions with `@tool` in the `run.py` file of their MCP server. When the MCP server starts, it dynamically loads and registers these decorated functions, making them available to agents and other MCP clients.

sasram.mcp.tool

tool Decorator Object

The decorator used to register a Python function as an MCP tool in a Retrieval Agent Manager code MCP server template.

```
python
from sasram.mcp import tool
```

`tool` is an alias of `mcp.tool` from the FastMCP library.

When you decorate a function with `@tool`, the function is exposed to MCP clients as a callable tool. The function name, type hints, and docstring are used to define the tool metadata and argument schema.

How It Works

IMPORTANT Tools must be defined in the `run.py` file of your code MCP server template.

- 1 Import the `tool` from the `sasram.mcp` file.
- 2 Decorate a function with `@tool`.
- 3 Save, publish, and instantiate an MCP tool server from the code MCP server template.
- 4 Register and use the tools with your Retrieval Agent Manager agents.

Example: Registering a Tool with `@tool`

The following example code performs these actions:

- Registers `summarize_text` as an MCP tool when the module is imported
- Exposes `text` and `max_words` as typed inputs
- Exposes the return type as the tool output type
- Uses the function docstring as tool description and parameter help

```
python
from sasram.mcp import tool

@tool
def summarize_text(text: str, max_words: int = 50) -> str:
    """
    Summarize input text to approximately max_words.

    Args:
        text: Input text to summarize.
        max_words: Maximum number of words in the summary.

    Returns:
        A shortened summary string.
    """
    words = text.split()
    return " ".join(words[:max_words])
```

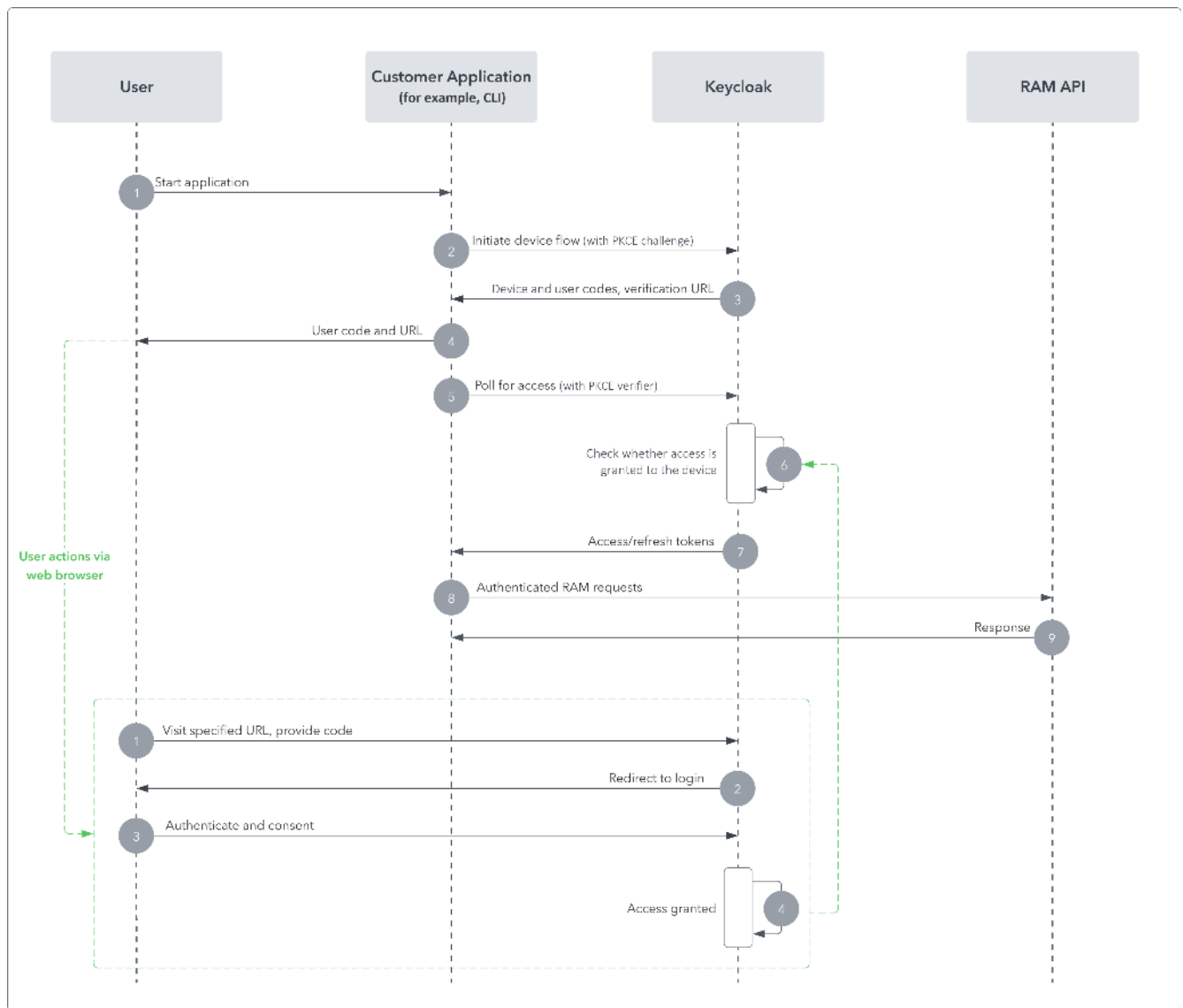
Integrating the Retrieval Agent Manager API with Applications

This section provides an overview of the API access methods for SAS Retrieval Agent Manager. The Retrieval Agent Manager API enables users to interact with SAS Retrieval Agent Manager through other applications.

Authentication Flows

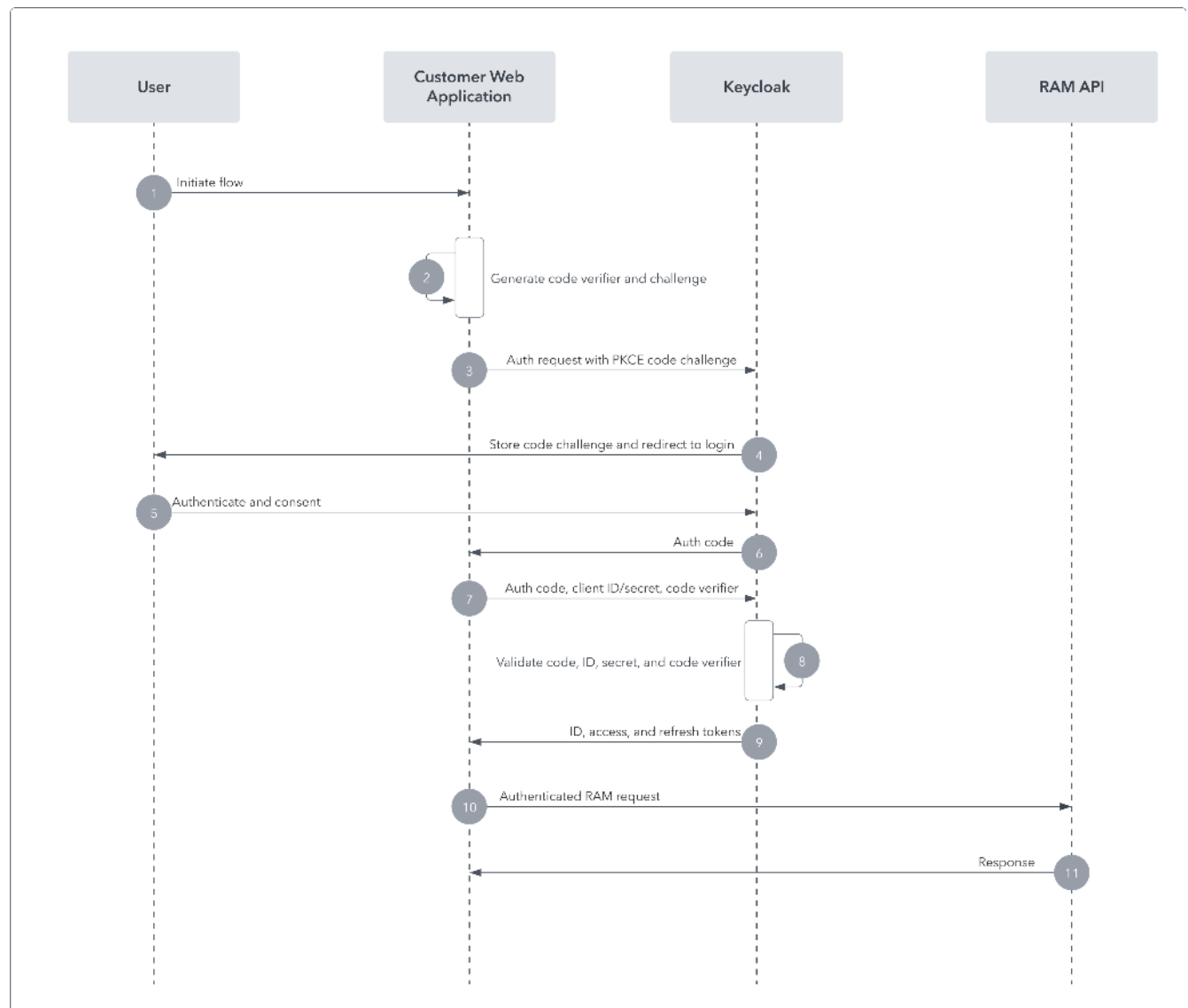
The Retrieval Agent Manager API uses OAuth 2.0 and OpenID Connect (OIDC) standards for authentication and authorization. When accessing the Retrieval Agent Manager API, clients must authenticate using one of the supported OAuth 2.0 flows.

The Retrieval Agent Manager API supports two authentication flows. The first supported flow is the device authorization flow. The device authorization flow is designed for devices with limited input capabilities or no web browser, such as IoT devices, command-line applications, or native applications. The user is provided with a code and a URL to visit via a web browser to authenticate and authorize the application. If necessary, the user can authenticate and authorize the application from another device, such as a mobile phone. This flow is ideal for scenarios where redirect-based authentication is not feasible.



Device Authorization Flow with Proof Key for Code Exchange (PKCE)

The other supported flow is the authorization code flow. This flow is used for user authentication where the user grants permission to the client application to access resources on their behalf. This flow involves redirecting the user to an authorization server to obtain an authorization code. The authorization code is then exchanged for an access token. A web browser is required to complete this flow, making it suitable for web applications. Allowed post-authentication redirect URIs must be pre-registered and carefully managed to ensure security.



Authorization Code Flow with Proof Key for Code Exchange (PKCE)

For both authentication flows, the Retrieval Agent Manager API requires that clients obtain access and refresh tokens from the identity provider that is configured for SAS Retrieval Agent Manager. The access token must be included in the authorization header of each API request that uses the bearer scheme. The refresh token can be used to obtain new access tokens when the current access token expires.

Choosing the Right Flow

When selecting an authentication flow for the Retrieval Agent Manager API, consider the following factors.

- **User Interaction**

If the application requires user interaction and operates in a web environment, the authorization code flow is appropriate. For applications running on devices without browsers or with limited input capabilities, the device code flow is more suitable.

- **Security Requirements**

Evaluate the security requirements of your application and choose the flow that best aligns with your security policies and practices. Each flow has its own security considerations, so ensure that you understand the implications of each option. For more details, see [Best Current Practice for OAuth 2.0 Security from the IETF](#).

■ Ease of Implementation

Consider the complexity of implementing each flow in your application. The authorization code flow might require more setup due to redirect handling. The device code flow might involve additional steps to handle polling for user authorization. Both flows are well-documented, industry-standard, and supported by various libraries, so choose the one that best fits your development environment and expertise.

By carefully evaluating these factors, you can select the most appropriate authentication flow for your needs when accessing the Retrieval Agent Manager API.

Required Configuration

For both the authorization code flow and device code flow, SAS Retrieval Agent Manager pre-configures a public client in the identity provider with client ID `sas-ram-api`. According to current best practices for public clients, this client is set up to support both flows and enforces the use of Proof Key for Code Exchange (PKCE) in both flows to enhance security. Ensure that your application implements PKCE when using either flow.

The device code flow requires no additional configuration and can use the default client without modification. The authorization code flow requires that redirect URIs be pre-registered with the identity provider to ensure that tokens are sent only to trusted locations. By default, the public client is configured to allow redirects to paths under the SAS Retrieval Agent Manager host. If you expect your application to run under a different host or domain, you need to add the appropriate redirect URIs to the client configuration in the identity provider.

Preconfigured Python Packages for Agents and Custom Sources

The following table lists the preconfigured Python packages that can be uploaded on the **Code** tab when adding an agent or a custom source. For agents, the packages enable actions such as executing Python scripts for multi-step workflows, calling APIs, and handling data transformations and custom logic. For custom sources, the packages enable you to integrate external data repositories or specialized content pipelines.

Package	Version
aiofiles	24.1.0
annotated-types	0.7.0

Package	Version
anyio	4.10.0
argon2-cffi	25.1.0
argon2-cffi-bindings	25.1.0
arrow	1.3.0
asttokens	3.0.0
async-lru	2.0.5
async-property	0.2.2
attrs	25.3.0
babel	2.17.0
beautifulsoup4	4.13.5
bleach	6.2.0
certifi	2025.8.3
cffi	2.0.0
charset-normalizer	3.4.3
click	8.2.1
comm	0.2.3
contourpy	1.3.3
coverage	7.10.6
cryptography	45.0.7
cycler	0.12.1
debugpy	1.8.16
decorator	5.2.1
defusedxml	0.7.1
deprecation	2.1.0

Package	Version
executing	2.2.1
fastapi	0.116.1
fastjsonschema	2.21.2
fonttools	4.59.2
fqdn	1.5.1
h11	0.16.0
httpcore	1.0.9
httptools	0.6.4
httpx	0.28.1
idna	3.10
iniconfig	2.1.0
ipykernel	6.30.1
ipython	9.5.0
ipython_pygments_lexers	1.1.1
ipywidgets	8.1.7
isoduration	20.11.0
jedi	0.19.2
Jinja2	3.1.6
json5	0.12.1
jsonpatch	1.33
jsonpointer	3.0.0
jsonschema	4.25.1
jsonschema-specifications	2025.9.1
jupyter	1.1.1

Package	Version
jupyter_client	8.6.3
jupyter-console	6.6.3
jupyter_core	5.8.1
jupyter-events	0.12.0
jupyter-lsp	2.3.0
jupyter_server	2.17.0
jupyter_server_terminals	0.5.3
jupyterlab	4.4.7
jupyterlab_pygments	0.3.0
jupyterlab_server	2.27.3
jupyterlab_widgets	3.0.15
jwtcrypto	1.5.6
kiwisolver	1.4.9
langchain-core	0.3.74
langsmith	0.4.27
lark	1.2.2
license_manager	1.1.0
MarkupSafe	3.0.2
matplotlib	3.10.6
matplotlib-inline	0.1.7
mistune	3.1.4
nbclient	0.10.2
nbconvert	7.16.6
nbformat	5.10.4

Package	Version
nest-asyncio	1.6.0
notebook	7.4.5
notebook_shim	0.2.4
numpy	2.3.3
orjson	3.11.3
overrides	7.7.0
packaging	25.0
pandas	2.3.2
pandocfilters	1.5.1
parso	0.8.5
pexpect	4.9.0
pillow	11.3.0
pip	25.2
platformdirs	4.4.0
pluggy	1.6.0
prometheus_client	0.22.1
prompt_toolkit	3.0.52
psutil	7.0.0
ptyprocess	0.7.0
pure_eval	0.2.3
pycparser	2.23
pydantic	2.11.7
pydantic_core	2.33.2
pydantic-settings	2.10.1

Package	Version
Pygments	2.19.2
PyJWT	2.10.1
pyparsing	3.2.3
pytest	8.4.2
pytest-cov	6.3.0
python-dateutil	2.9.0.post0
python-dotenv	1.1.1
python-json-logger	3.3.0
python-keycloak	5.5.0
pytz	2025.2
PyYAML	6.0.2
pyzmq	27.1.0
referencing	0.36.2
requests	2.32.5
requests-toolbelt	1.0.0
rfc3339-validator	0.1.4
rfc3986-validator	0.1.1
rfc3987-syntax	1.1.0
rpds-py	0.27.1
sasram	0.0.1
sasviya-cardinality	0.1.50
sasviya-core	0.20.132
sasviya-cv	0.16.0
sasviya-decomposition	0.6.2

Package	Version
sasviya-gmm	0.1.38
sasviya-linear-model	0.4.3
sasviya-mlcore	0.2.4
sasviya-svm	0.3.3
sasviya-tree	0.5.3
Send2Trash	1.8.3
setuptools	80.9.0
six	1.17.0
sniffio	1.3.1
soupsieve	2.8
stack-data	0.6.3
starlette	0.47.3
tenacity	9.1.2
terminado	0.18.1
tinycss2	1.4.0
tornado	6.5.2
traitlets	5.14.3
types-python-dateutil	2.9.0.20250822
typing_extensions	4.15.0
typing-inspection	0.4.1
tzdata	2025.2
uri-template	1.3.0
urllib3	2.5.0
uvicorn	0.35.0

Package	Version
uvloop	0.21.0
watchfiles	1.1.0
wcwidth	0.2.13
webcolors	24.11.1
webencodings	0.5.1
websocket-client	1.8.0
websockets	15.0.1
wheel	0.38.4
widetsnbextension	4.0.14
zstandard	0.24.0